

EXPERIMENT NO. 4

Aim

To study the structure of Solidity programs, basic Solidity constructs, and the compilation and deployment environment, and to gain hands-on experience in creating and interacting with smart contracts using Remix IDE.

Theory

Solidity is a high-level programming language used to develop smart contracts on Ethereum-compatible blockchain platforms. A smart contract is a program stored on a blockchain that contains predefined rules and functions that can be executed by users or other contracts.

A Solidity program generally consists of a **license declaration, pragma statement, contract declaration, state variables, constructors, modifiers and functions.**

1. Primitive Data Types and Variables

Solidity provides several basic data types:

- **uint / int:** Used for unsigned and signed integers.
- **bool:** Represents **true** or **false**.
- **address:** Stores an Ethereum account or smart contract address.
- **string:** Stores textual data.
- **bytes:** Stores binary data.

Variables in Solidity can be classified as:

- **State variables:** Stored permanently on the blockchain.
- **Local variables:** Exist temporarily during function execution.
- **Global variables:** Predefined blockchain-related variables such as **msg.sender**, **msg.value** and **block.timestamp**.

2. Functions

Functions contain the executable logic of a smart contract.

A **pure** function does not read or modify the contract's state.

```
function add(uint a, uint b)
    public
    pure
```

```
    returns(uint)
{
    return a + b;
}
```

A **view** function can read state variables but cannot modify them.

```
function getValue()
    public
    view
    returns(uint)
{
    return value;
}
```

3. Function Inputs and Outputs

Functions can accept input parameters and return one or more values.

For example:

```
function multiply(uint a, uint b)
    public
    pure
    returns(uint)
{
    return a * b;
}
```

Here, **a** and **b** are inputs and the multiplication result is the output.

4. Visibility

Solidity provides four major visibility types:

- **public:** Can be accessed internally and externally.
- **private:** Can only be accessed within the same contract.
- **internal:** Can be accessed within the contract and derived contracts.
- **external:** Intended to be called from outside the contract.

5. Modifiers

Modifiers are reusable pieces of code that can restrict or modify the execution of functions. They are commonly used for access control.

Example:

```
modifier onlyOwner() {  
    require(msg.sender == owner, "Not owner");  
    _;  
}
```

A function using this modifier can only be executed when the specified condition is satisfied.

6. Constructors

A constructor is executed when a smart contract is deployed. It is generally used to initialize state variables.

Example:

```
constructor() {  
    owner = msg.sender;  
}
```

7. Control Flow

Solidity supports standard control-flow statements such as:

- `if-else`
- `for`
- `while`
- `do-while`

These statements are used for conditional and repeated execution of code. Loops must be used carefully in smart contracts because their execution consumes gas.

8. Data Structures

Solidity supports several data structures.

Arrays store ordered collections of values.

```
uint[] public numbers;
```

Mappings store key-value pairs.

```
mapping(address => uint) public balances;
```

Structs allow multiple related values to be grouped together.

```
struct Player {  
    string name;  
    uint score;
```

```
}
```

Enums define a fixed set of named values.

```
enum Status {  
    Pending,  
    Active,  
    Closed  
}
```

9. Data Locations

Solidity mainly uses three data locations:

- **storage:** Persistent data stored on the blockchain.
- **memory:** Temporary data that exists during function execution.
- **calldata:** Temporary and read-only data generally used for external function parameters.

10. Ether, Wei and Transactions

Ether (ETH) is the native currency of the Ethereum network.

The smallest unit of Ether is **Wei**:

1 Ether = 10^{18} Wei

Transactions are used to interact with smart contracts or transfer Ether.

A function that receives Ether generally needs to be declared **payable**.

11. Gas and Gas Price

Gas represents the computational work required to execute an Ethereum transaction or smart-contract operation.

The transaction cost is conceptually related to:

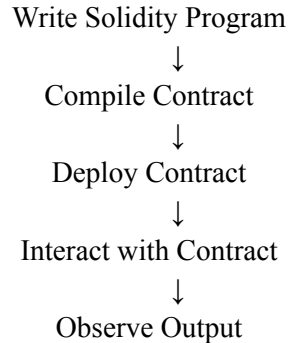
Transaction Fee = Gas Used × Gas Price

Gas encourages efficient smart-contract design because computationally expensive operations require more resources.

12. Remix IDE

Remix IDE is a browser-based development environment for writing, compiling, deploying and testing Solidity smart contracts.

The basic smart-contract development process is:



For this experiment, the **Remix VM** is used as the testing/deployment environment. The experiment requires exploring the Solidity Basics Course available through Remix LearnETH and completing its 19 assignments.

Procedure

Step 1: Open Remix IDE

Open the Remix IDE using:

<https://remix.ethereum.org/>

Navigate to the Solidity/LearnETH section and open the **Solidity Basics Course**.

Step 2: Explore the Solidity Basics Course

Study the concepts and examples provided in the LearnETH Solidity Basics Course.

The course contains **19 assignments/tutorials** that demonstrate different Solidity programming concepts.

Step 3: Open Tutorial 1

Open **Tutorial 1** from the Solidity Basics Course.

Study the provided Solidity smart contract and understand its variables and functions.

Step 4: Compile the Smart Contract

1. Open the **Solidity Compiler** panel in Remix IDE.
2. Select a compiler version compatible with the contract.
3. Compile the Solidity program.
4. Verify that there are no compilation errors.
5. Record the successful compilation output.

Step 5: Deploy the Smart Contract

1. Open **Deploy & Run Transactions**.
2. Select the **Remix VM** environment.
3. Select the appropriate account.
4. Select the compiled contract.
5. Click **Deploy**.
6. Verify that the contract appears under **Deployed Contracts**.

Step 6: Execute the **get** Function

After deployment, interact with the deployed contract.

Click the **get** function and observe the value returned by the contract.

Record the output.

Step 7: Execute the **increment** Function

Click the **increment** function.

This performs a state-changing operation on the smart contract.

After executing it, call the **get** function again and observe the updated value.

Step 8: Execute the **decrement** Function

Click the **decrement** function.

After the transaction is completed, execute the **get** function again and observe the updated value.

Step 9: Complete Tutorials 2–19

Continue with the remaining assignments in the Solidity Basics Course:

Tutorial 2
Tutorial 3
Tutorial 4
Tutorial 5
Tutorial 6
Tutorial 7
Tutorial 8
Tutorial 9
Tutorial 10
Tutorial 11
Tutorial 12

Tutorial 13
Tutorial 14
Tutorial 15
Tutorial 16
Tutorial 17
Tutorial 18
Tutorial 19

For each tutorial:

1. Open the assignment.
2. Study the Solidity program.
3. Compile the program.
4. Deploy the contract using Remix VM where required.
5. Execute the functions provided by the tutorial.
6. Observe and record the outputs.
7. Take the required screenshots for documentation.

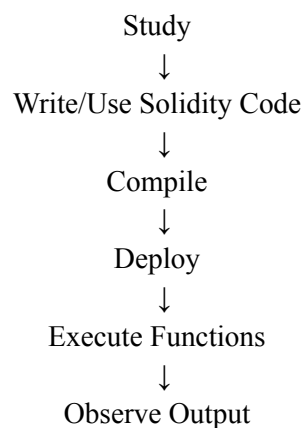
Step 10: Verify Smart Contract Interaction

For each deployed contract, interact with its available functions through the **Deployed Contracts** section.

Read-only functions can be used to retrieve information, while state-changing functions are executed as transactions and consume gas.

Step 11: Complete the 19 Assignments

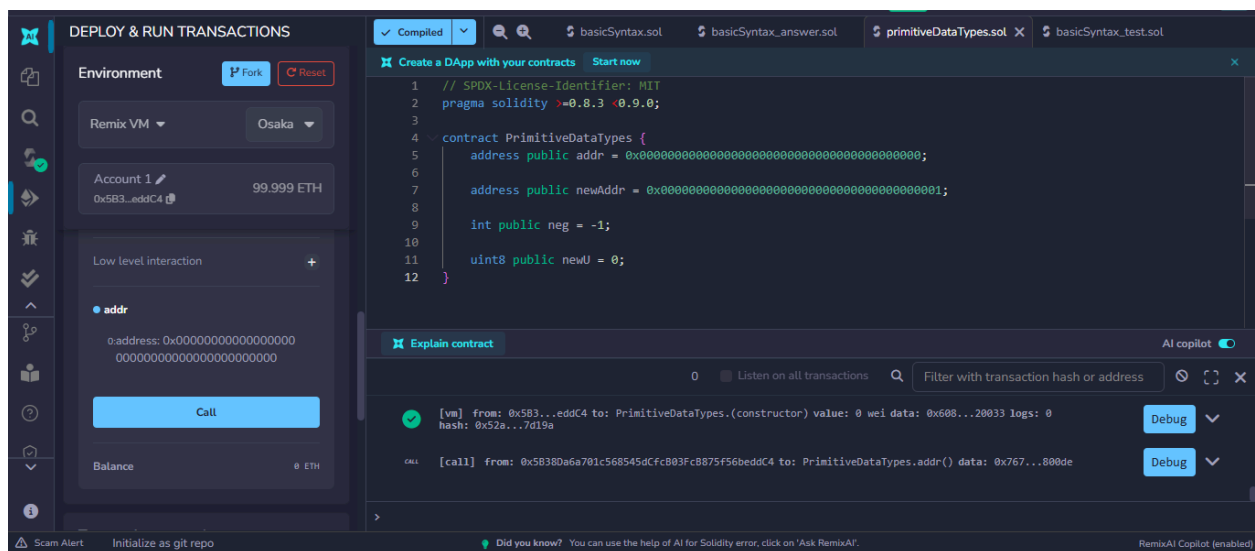
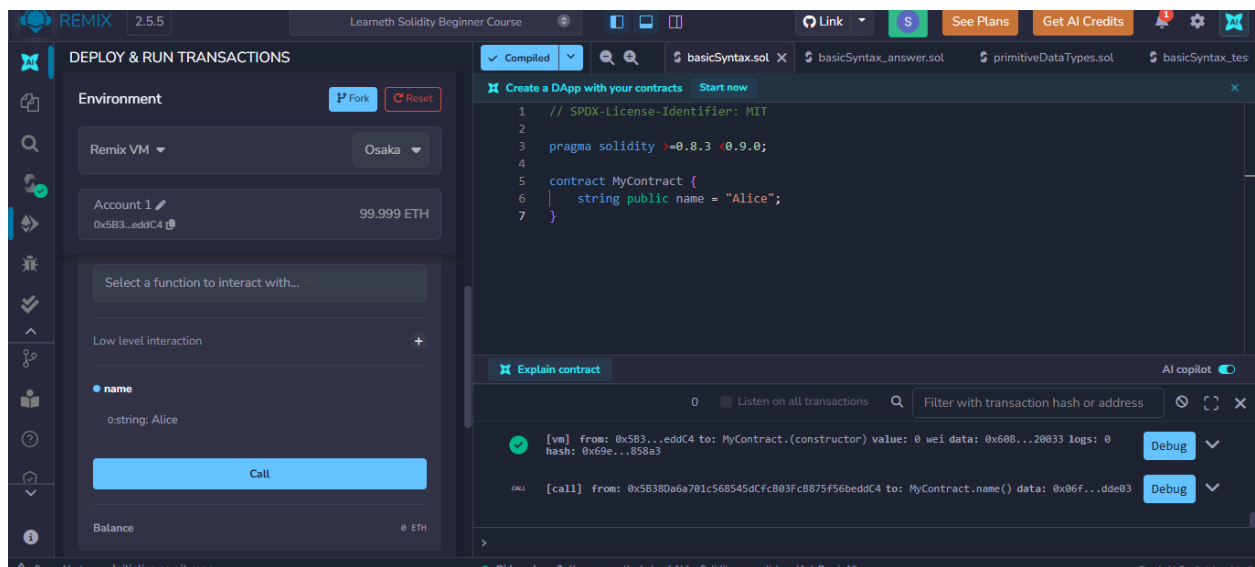
All 19 assignments provided in the LearnETH Solidity Basics Course are completed by following the same process of:

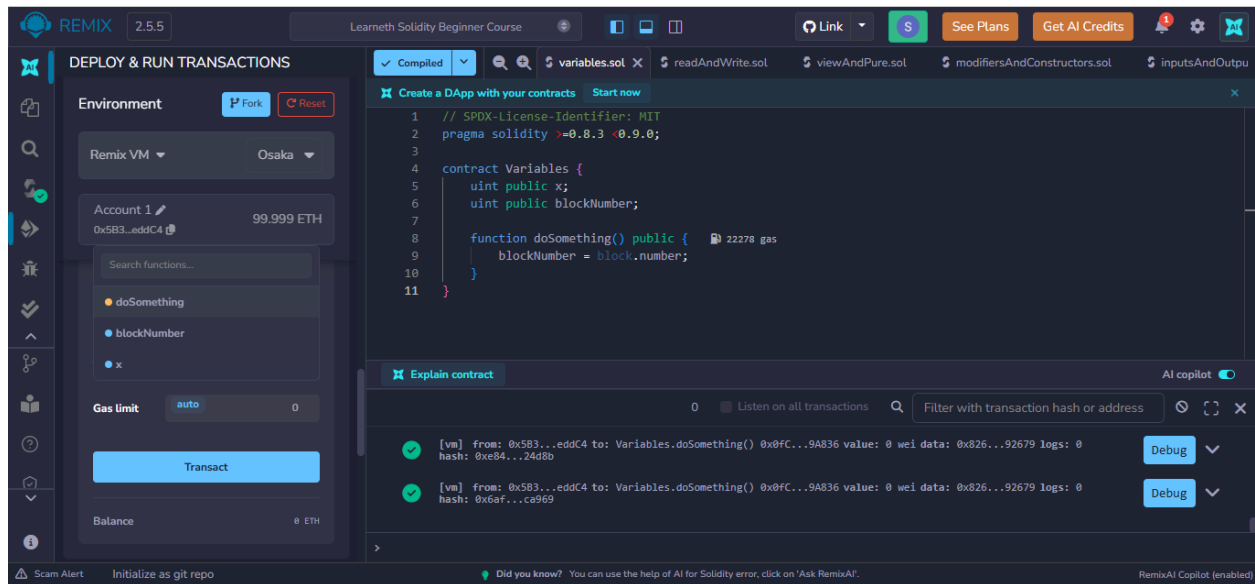
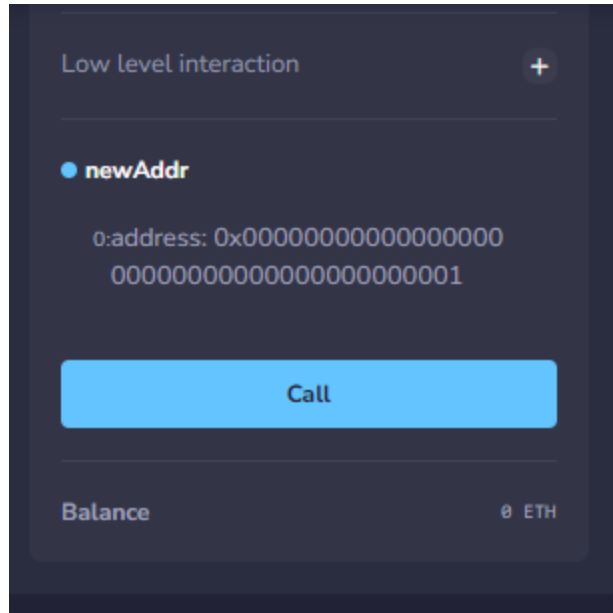


Conclusion

The fundamentals of Solidity programming were successfully studied and implemented using the Remix IDE. The experiment provided practical understanding of Solidity program structure, primitive data types, variables, functions, function inputs and outputs, visibility, modifiers, constructors, control-flow statements, arrays, mappings, structs, enums and data locations.

The concepts of Ether, Wei, gas, gas price and transactions were also explored. The Solidity programs were compiled and deployed using the Remix VM, and the smart-contract functions were executed and tested successfully. The 19 assignments from the Solidity Basics Course provided hands-on experience in developing and interacting with smart contracts, thereby strengthening the understanding of Solidity and blockchain-based smart-contract development.





REMIX 2.5.5 Learneth Solidity Beginner Course

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, sanket 99.999 ETH

Low level interaction: setX uint256, _x (uint256)

Value: 0, Gas limit: auto

Transact

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7
8     // You need to send a transaction to write to a state variable.
9     function set(uint _num) public { 22514 gas
10         num = _num;
11     }
12
13     // You can read from a state variable without sending a transaction.
14     function get() public view returns (uint) { 2453 gas
15         return num;
16     }
17 }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x583...eddC4 to: Variables.doSomething() 0x8fC...9A836 value: 0 wei data: 0x826...92679 logs: 0 hash: 0x6af...ca969 Debug

[vm] from: 0x583...eddC4 to: ReadAndWrite.(constructor) value: 0 wei data: 0x60B...20033 logs: 0 hash: 0x96e...171f0 Debug

Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'.

REMIX 2.5.5 Learneth Solidity Beginner Course

DEPLOY & RUN TRANSACTIONS

Environment: Solidity compiler M, Osaka, sanket 99.999 ETH

SimpleStorage 0xD4F...2cbee 6min ago

Functions: Select a function to interact with...

Low level interaction: num, 0:uint256: 50

Transact

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {  PUSH1 costs 3 gas - this line costs 96 gas - 2978936 gas left
5     // State variable to store a number
6     uint public num;
7
8     // You need to send a transaction to write to a state variable.
9     function set(uint _num) public { 22514 gas
10         num = _num;
11     }
12
13     // You can read from a state variable without sending a transaction.
14     function get() public view returns (uint) { 2453 gas
15         return num;
16     }
17 }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

CALL [call] from: 0x58380a6a701c568545dcFc803Fc8875F56beddC4 to: SimpleStorage.num() data: 0x4e7...0b1dc Debug

CALL [call] from: 0x58380a6a701c568545dcFc803Fc8875F56beddC4 to: SimpleStorage.num() data: 0x4e7...0b1dc Debug

Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'.

REMIX 2.5.5 Learneth Solidity Beginner Course

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, Account 1 (0x5B3...eddC4), 99.999 ETH

Functions: Select a function to interact with...

Low level interaction: 0:uint256: 1

Call

Compiled: tax_test.sol, variables.sol, readAndWrite.sol, viewAndPure.sol, modifiersAndConstructors.sol

Create a DApp with your contracts Start now

```
9 return x + y;
10 }
11
12 // Promise not to modify or read from the state.
13 function add(uint i, uint j) public pure returns (uint) {
14     return i + j;
15 }
16 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

0x5B3...eddC4 to: ViewAndPure.constructor() value: 0 wei data: 0x608...20033 logs: 0 hash: 0x774...ec915

0x5B38Da6a701c56854dCfc8B3Fc8875F56beddC4 to: ViewAndPure.x() data: 0x8c5...5699c

Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI' RemixAI Copilot (enabled)

REMIX 2.5.5 Learneth Solidity Beginner Course

DEPLOY & RUN TRANSACTIONS

Environment: Remix VM, Osaka, Account 1 (0x5B3...eddC4), 99.999 ETH

Functions: Select a function to interact with...

Low level interaction: addToX uint256, 5, Calldata, Parameters, 0:uint256: 6

Call

Balance 0 ETH

Compiled: tax_test.sol, variables.sol, readAndWrite.sol, viewAndPure.sol, modifiersAndConstructors.sol

Create a DApp with your contracts Start now

```
9 return x + y;
10 }
11
12 // Promise not to modify or read from the state.
13 function add(uint i, uint j) public pure returns (uint) {
14     return i + j;
15 }
16 }
```

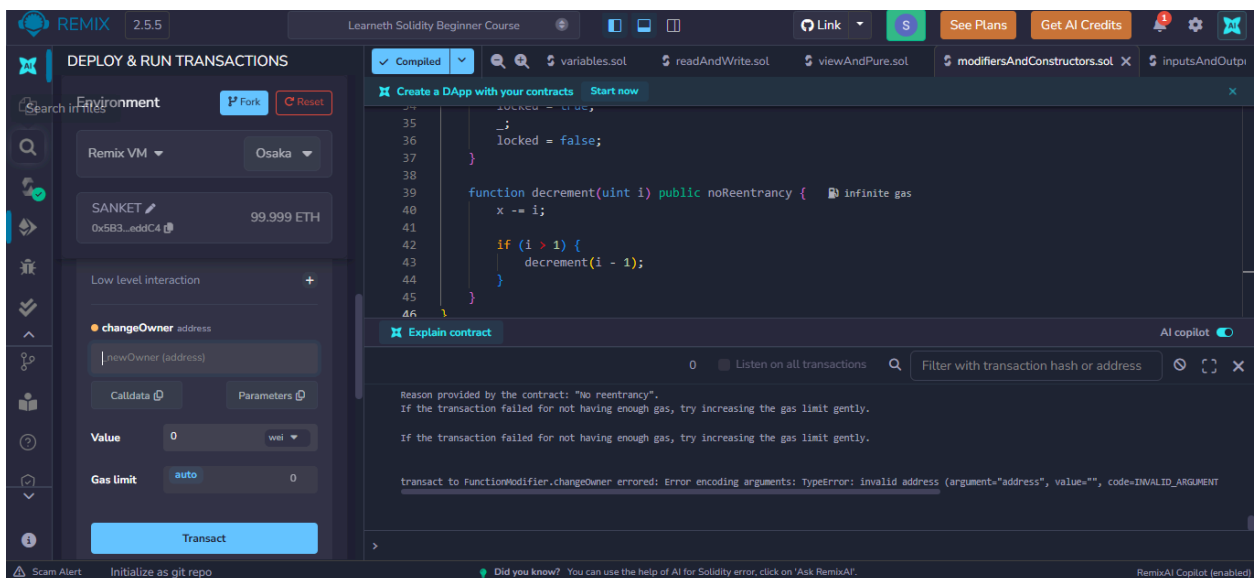
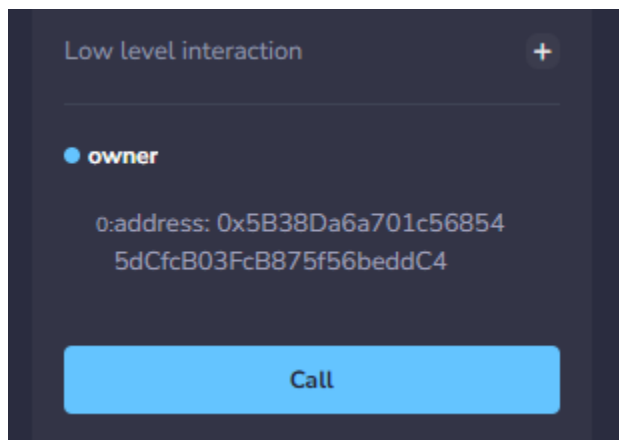
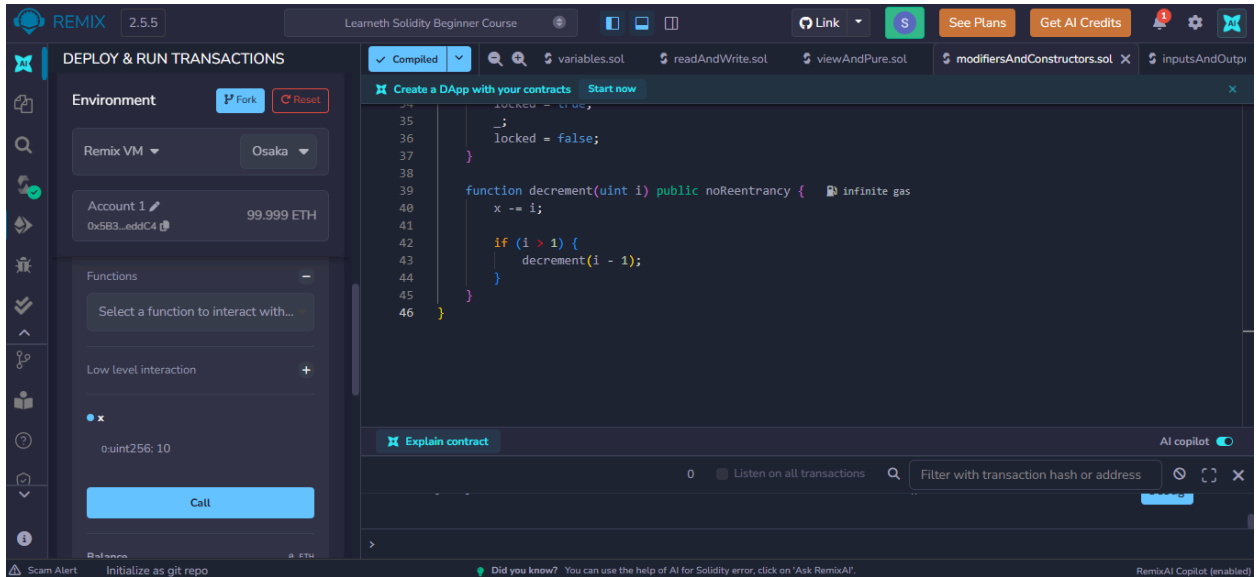
Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

0x5B38Da6a701c56854dCfc8B3Fc8875F56beddC4 to: ViewAndPure.x() data: 0x8c5...5699c

0x5B38Da6a701c56854dCfc8B3Fc8875F56beddC4 to: ViewAndPure.addToX(uint256) data: 0x596...00005



REMIX2.5.5

Learneth Solidity Beginner Course

Link

See Plans

Get AI Credits

DEPLOY & RUN TRANSACTIONS

Environment

ForkReset

Remix VM

Osaka

SANKET

0x5B3...eddC4

99.999 ETH

Functions

Select a function to interact with...

Low level interaction

externalFunc

0:string: external function called

Call

Balance

0 ETH

Compiled

Create a DApp with your contracts

Start now

35function testInternalFunc() public pure override returns (string memory) { infinite gas

36return internalFunc();

37}

38}

viewAndPure.sol

modifiersAndConstructors.sol

inputsAndOutputs.sol

visibility.sol

Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or address

transact to FunctionModifier.changeOwner errored: Error encoding arguments: TypeError: invalid address (argument="address", value="", code=INVALID_ARGUMENT)

[vm] from: 0x5B3...eddC4 to: Base.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x4ac...9e77b

Debug

[call] from: 0x5B380a6a701c568545dcFcB03Fc8875F56beddC4 to: Base.externalFunc() data: 0x42b...148aa

Debug

Scam AlertInitialize as git repo

Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'.

RemixAI Copilot (enabled)

REMIX2.5.5

Learneth Solidity Beginner Course

Link

See Plans

Get AI Credits

DEPLOY & RUN TRANSACTIONS

Environment

ForkReset

Remix VM

Osaka

SANKET

0x5B3...eddC4

99.999 ETH

Functions

Select a function to interact with...

Low level interaction

publicFunc

0:string: public function called

Call

Balance

0 ETH

Compiled

Create a DApp with your contracts

Start now

35function testInternalFunc() public pure override returns (string memory) { infinite gas

36return internalFunc();

37}

38}

viewAndPure.sol

modifiersAndConstructors.sol

inputsAndOutputs.sol

visibility.sol

Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or address

[vm] from: 0x5B3...eddC4 to: Base.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x4ac...9e77b

Debug

[call] from: 0x5B380a6a701c568545dcFcB03Fc8875F56beddC4 to: Base.externalFunc() data: 0x42b...148aa

Debug

[call] from: 0x5B380a6a701c568545dcFcB03Fc8875F56beddC4 to: Base.publicFunc() data: 0x4b7...3383f

Debug

Scam AlertInitialize as git repo

Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'.

RemixAI Copilot (enabled)